



UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING
CEF440: INTERNET PROGRAMMING AND MOBILE PROGRAMMING

**PROJECT TITLE: DESIGN AND IMPLEMENTATION OF AN ADAPTIVE E-
LEARNING PLATFORM BASED ON QUALITY OF EXPERIENCE FOR
INCLUSIVE ONLINE EDUCATION IN LOW – RESOURCE SETTINGS.**

TASK 4

Group 15 Report

COURSE INSTRUCTOR: Dr Nkemeni Valery

NAME	MATRICULE
NJINYA NGECHA RYAN BROWN	FE23A116
KARL JONAS WENGE ACHA	FE23A072
BRIANNA TEBESICK TSAMO	FE23A028
KELSEY NJOCK-AWOH NJOCK OBEN	FE23A073

23th May, 2026

TABLE OF CONTENT

INTRODUCTION	3
1. USE CASE DIAGRAM	3
1.1 Actor Classifications	3
1.2 Detailed Use Case Descriptions	4
2. CLASS DIAGRAM	7
2.1 Core Entities and Attributes	7
2.2 Associative (Transactional) Entities	8
3. DEPLOYMENT DIAGRAM	8
3.1 Client Nodes	8
3.2 Cloud Infrastructure Nodes	9
3.3 Communication Protocols	9
4. SEQUENCE DIAGRAMS	9
4.1 Execution Flow: Learner Accessing Content (Online vs. Offline)	14
5. DATA FLOW DIAGRAM (LEVEL 1)	15
5.1 Overview	15
5.2 Data Stores	15
5.3 Process Descriptions	15
5.4 Key Internal Data Flows	16
6. CONTEXT DIAGRAM DESCRIPTION	17
6.1 Overview	17
6.2 External Entities and Their Data Flows	17
6.3 Design Decisions	18
CONCLUSION	18

INTRODUCTION

This report presents the System Modelling and Design phase (Task 4) for the project entitled "Design and Implementation of an Adaptive E-learning Platform based on Quality of Experience for Inclusive Online Education in Low-resource Settings."

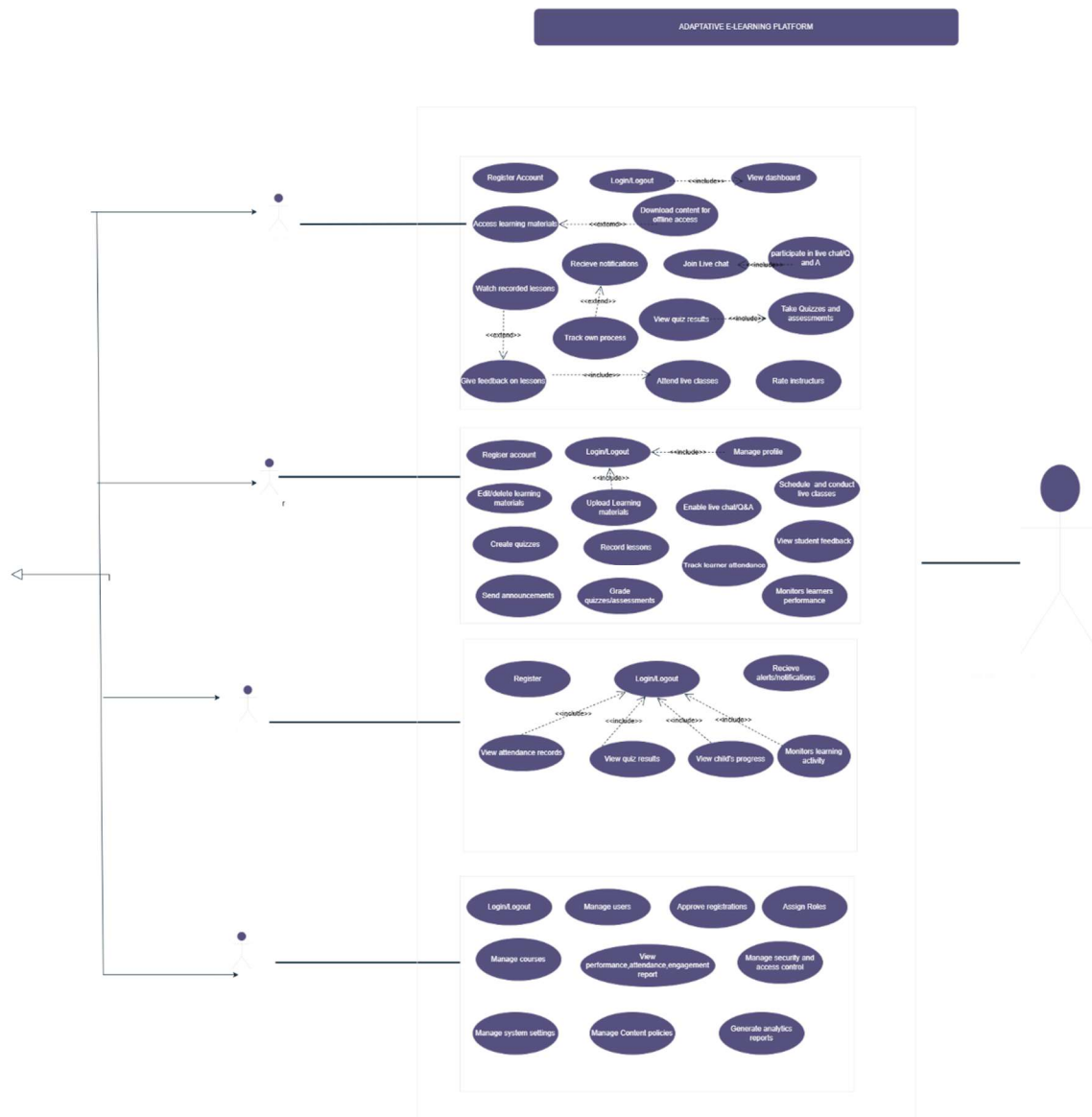
Following the requirement analysis established in Task 3, this phase translates the identified functional and non-functional requirements into comprehensive architectural blueprints. Using the Unified Modeling Language (UML) and structured systems analysis techniques, this report details the system's structural and behavioural models. The design strictly adheres to the technological constraints of a Progressive Web App (PWA) architecture, utilising a React frontend, Node.js/Express backend, and PostgreSQL database to ensure offline capability, low-bandwidth optimization, and adaptive content delivery.

1. USE CASE DIAGRAM

The Use Case diagram models the system's behavioural requirements from the perspective of its users. The actors are classified based on their level of interaction with the core system processes.

1.1 Actor Classifications

- **Primary Actors (Active Initiators):**
 - **Learner:** Operates primarily via a mobile interface. Interacts with core learning materials, assessments, and offline syncing.
 - **Instructor:** Operates primarily via a laptop interface. Manages curriculum, uploads content, and tracks learner participation.
- **Secondary Actor (System Maintainer):**
 - **Administrator:** Ensures platform health, manages security, and provisions user roles.
- **Tertiary Actor (Passive Observer):**
 - **Parent:** Monitors academic progress and attendance without altering system state or data.



1.2 Detailed Use Case Descriptions

UC1: Log In

- **Actors:** Learner, Instructor, Parent, Administrator.
- **Description:** This use case allows registered users to securely authenticate themselves and access their respective role-based dashboards. The user inputs their registered email address and password.
- **Preconditions:** The user must have an active, registered account provisioned in the database.
- **Postconditions:** The system grants access and routes the user to the appropriate interface based on their role.
- **Relationships:** <<includes>> UC2 (Verify Credentials).

UC2: Verify Credentials

- **Actors:** None (Internal System Process).
- **Description:** An automated backend process that securely encrypts and validates the login credentials provided in UC1 against the PostgreSQL database. It issues a secure authentication token upon success.
- **Preconditions:** UC1 is initiated.

UC3: Access Adaptive Content

- **Actors:** Learner.
- **Description:** The learner browses and accesses course modules, video lectures, and text materials. The system dynamically evaluates the learner's current network bandwidth and device capabilities, adapting the media quality (e.g., serving a low-resolution video or text-only fallback) to optimize the Quality of Experience (QoE) and conserve mobile data.
- **Preconditions:** The learner is successfully logged in and actively enrolled in the selected course.
- **Postconditions:** Content is rendered on the learner's device without excessive buffering.

UC4: Take Quiz

- **Actors:** Learner.
- **Description:** The learner attempts assessments, quizzes, and examinations designed by the instructor. The system captures responses, automatically grades objective questions, and commits the final score to the database.
- **Preconditions:** The assessment is published and within its active due date.

UC5: Save / Cache Offline

- **Actors:** Learner.
- **Description:** The platform utilizes the PWA Service Worker to explicitly save course materials or pending assessment answers to the device's local storage (IndexedDB). This allows the learner to continue studying and complete tests during power outages or sudden internet disconnections.
- **Relationships:** <<extends>> UC3 and UC4. This is triggered conditionally if the network drops or the user manually opts to download for later use.

UC6: View Progress & Attendance

- **Actors:** Learner, Parent.
- **Description:** Allows learners to track their own academic performance metrics. It simultaneously allows parents to passively monitor their child's grades, completed modules, and attendance history.
- **Preconditions:** Data must be aggregated by the system. Parents must be cryptographically linked to their child's account profile.

UC7: Participate in Live Chat

- **Actors:** Learner, Instructor.
- **Description:** Facilitates real-time, text-based communication using WebSockets. Instructors can broadcast announcements or answer queries, while learners can ask for clarification during active, low-bandwidth text sessions.

UC8: Upload Learning Materials

- **Actors:** Instructor.
- **Description:** The instructor creates and structures course content. This includes

uploading text documents, compressing videos, and structuring quiz forms to the platform using their desktop/laptop interface.

UC9: Track Attendance

- **Actors:** Instructor.
- **Description:** The instructor reviews system-generated logs of learner activity to formally record attendance. The system intelligently accounts for both live session participation and offline module completion (which is registered retroactively once the learner's device reconnects and syncs).

UC10: Manage Courses

- **Actors:** Instructor.
- **Description:** The instructor manages the overarching curriculum. Actions include creating new courses, organising modules, updating outdated content, and enrolling/removing specific learners from their class rosters.

UC11: Manage Users & Roles

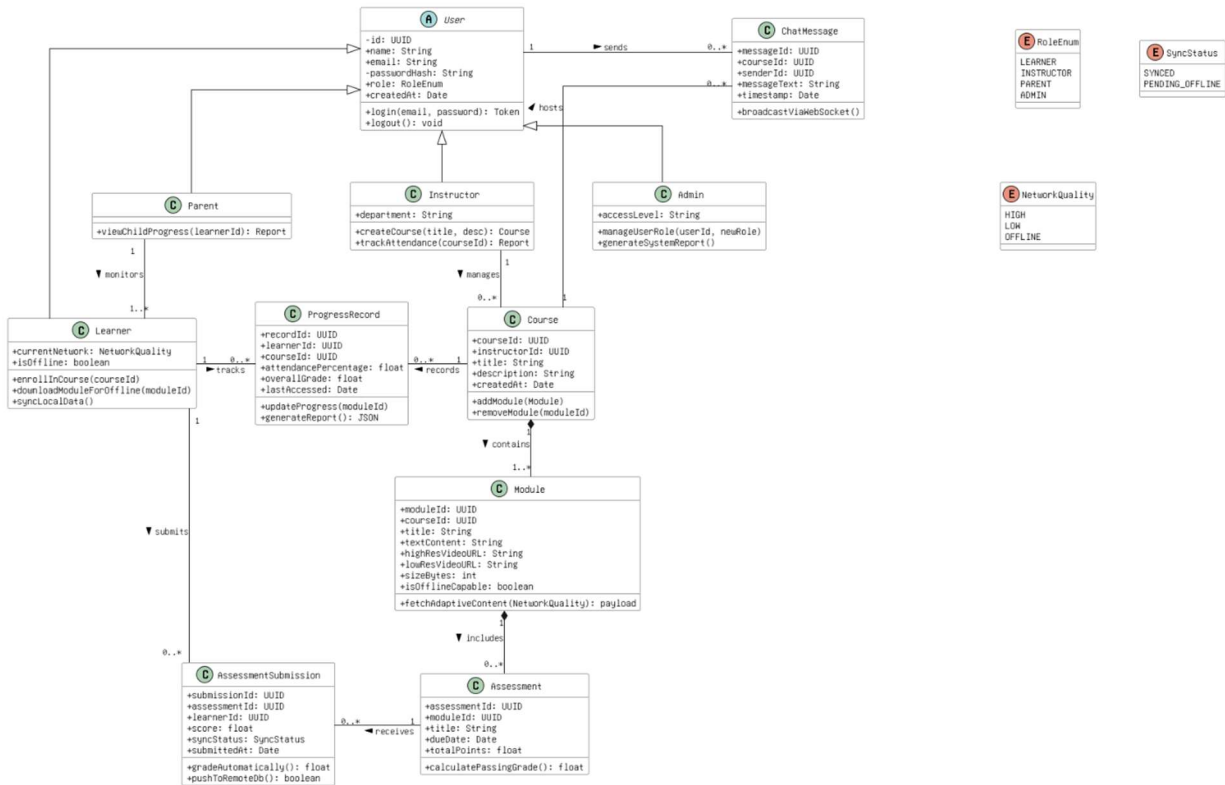
- **Actors:** Administrator.
- **Description:** The administrator handles account provisioning and lifecycle management. This includes registering new user profiles, resetting compromised passwords, and assigning strict access roles (Learner, Instructor, Parent, Admin) in the database.

UC12: Generate System Reports

- **Actors:** Administrator.
- **Description:** The administrator queries the database to generate platform-wide operational analytics, tracking overall system usage, active concurrent user counts, and platform health to ensure the infrastructure operates smoothly in the low-resource environment.

2. CLASS DIAGRAM

The Class Diagram models the static structure of the database and backend logic. It defines the entities, their strict data types, visibility modifiers, and relational database mappings required for PostgreSQL implementation.



2.1 Core Entities and Attributes

- **User (Abstract Base Class):** Centralizes shared attributes such as id (UUID), name (String), email (String), passwordHash (String - Private), and role (RoleEnum). It contains the core login() and logout() methods.
- **Learner (Concrete Class):** Inherits from User. Contains specific attributes like currentNetwork (NetworkQuality) and isOffline (boolean). Features methods such as downloadModuleForOffline() and syncLocalData() which directly support the PWA requirements.
- **Instructor (Concrete Class):** Inherits from User. Includes administrative attributes like department (String) and methods to createCourse() and trackAttendance().
- **Course & Module:** The Course class (courseId, title) aggregates the curriculum. A strict **Composition** relationship exists between Course and Module (moduleId, textContent, highResVideoURL, lowResVideoURL). The Module class contains the fetchAdaptiveContent() method to support dynamic QoE resolution switching.
- **Assessment:** Composed strictly within a Module. Tracks dueDate and totalPoints.

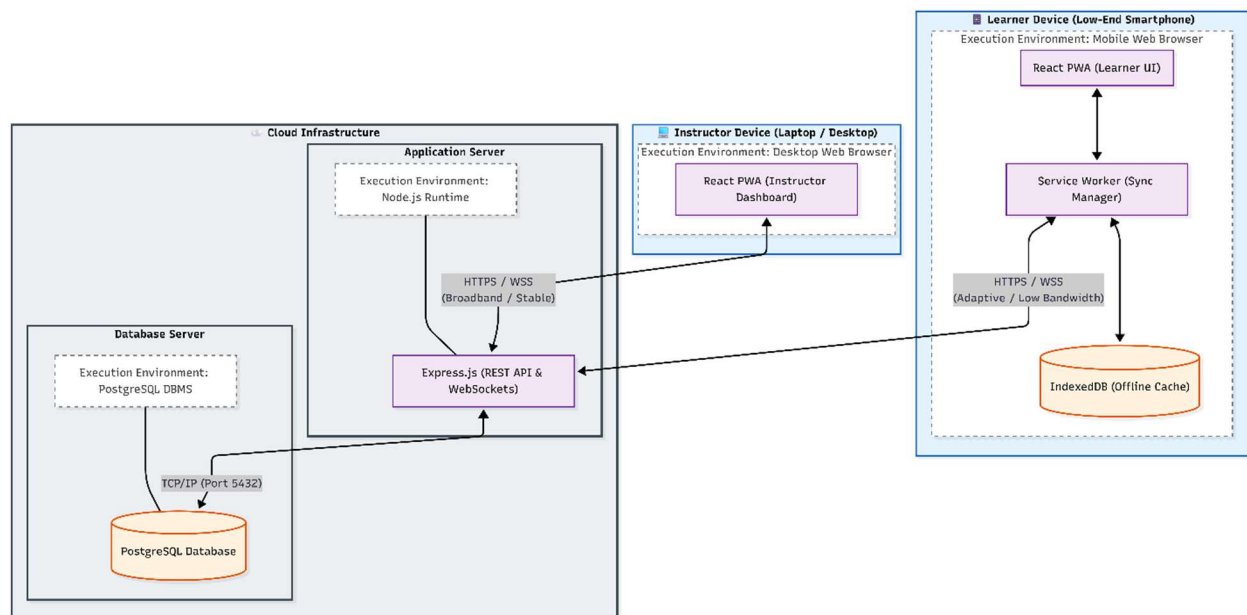
2.2 Associative (Transactional) Entities

To correctly map Many-to-Many relationships in PostgreSQL, associative entities are utilized:

- **ProgressRecord:** Sits between Learner and Course. It tracks attendancePercentage and overallGrade, linking the learnerId and courseId as foreign keys.
- **AssessmentSubmission:** Links Learner to Assessment. It tracks the score and, crucially, a syncStatus (SyncStatus Enum) which dictates whether the quiz was submitted online or is pending in the local offline cache.

3. DEPLOYMENT DIAGRAM

The Deployment Diagram physically maps the software artefacts (React, Node.js, PostgreSQL) to the hardware nodes and network environments, visualizing how the system operates under hardware and connectivity constraints.



3.1 Client Nodes

- **Learner Device (Low-End Smartphone):** The execution environment is the mobile web browser. It hosts the React PWA Learner UI. To combat power outages and internet instability, this node physically runs a **Service Worker** (Sync Manager) and an **Indexed DB** local database to cache content and queue assessment submissions offline.
- **Instructor Device (Laptop/Desktop):** The execution environment is the desktop web

browser, hosting the React PWA Instructor Dashboard, optimized for larger screen resolutions for administrative ease.

3.2 Cloud Infrastructure Nodes

- **Application Server:** A cloud-based virtual machine running the Node.js runtime. It hosts the Express.js framework which serves two purposes: a REST API for standard HTTP data fetching (adaptive content delivery) and a WebSocket Server (WSS) to handle low-latency real-time live chat.
- **Database Server:** A dedicated node running the PostgreSQL DBMS. It provides structured persistence for all relational data, safely insulated from the client tier.

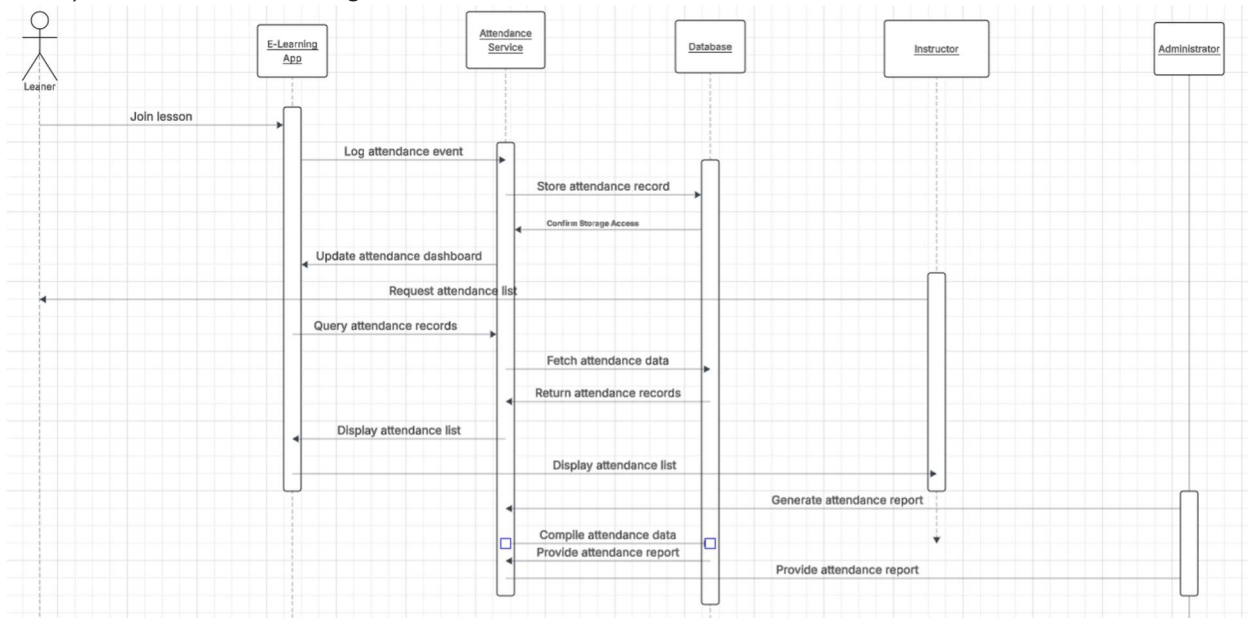
3.3 Communication Protocols

- The connection between the Client Devices and the Application Server utilizes **HTTPS and WSS**. For the Learner Node, this connection is strictly adaptive, heavily compressed, and fault-tolerant to handle low-bandwidth scenarios.
- The connection between the Application Server and the Database Server utilizes secure internal **TCP/IP (Port 5432)**.

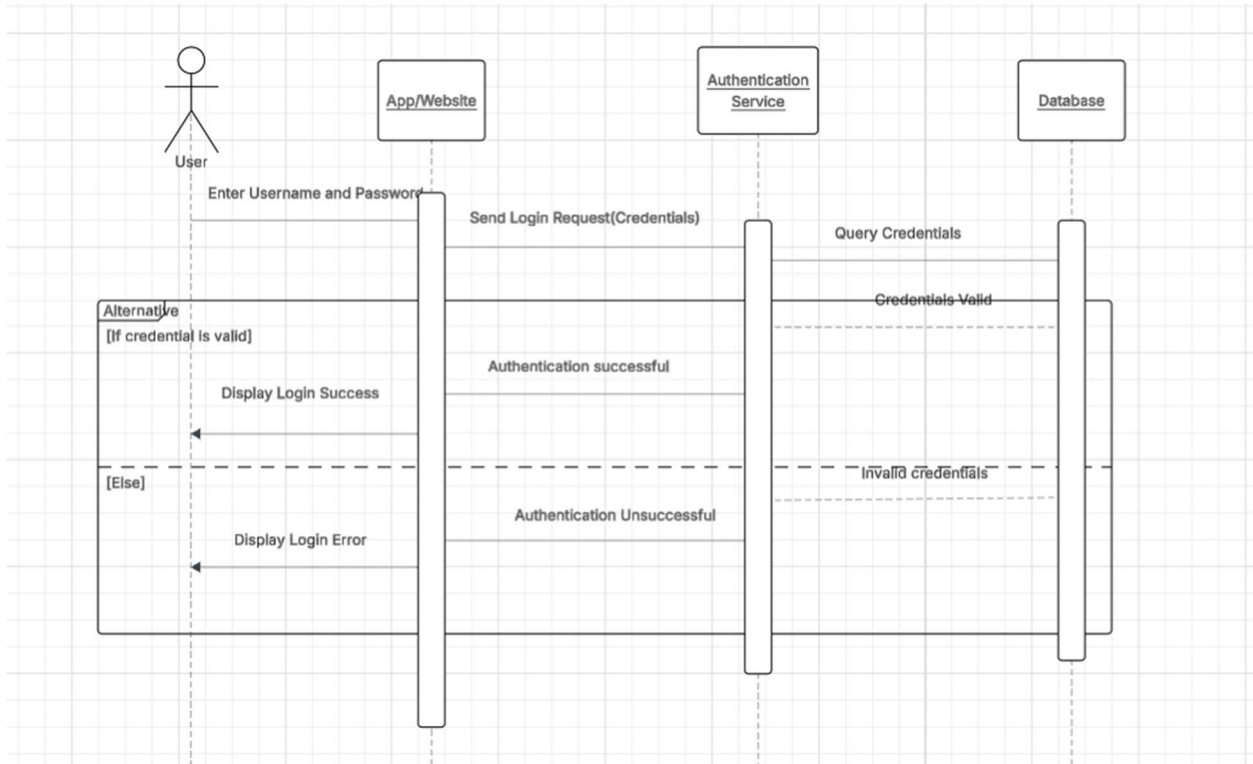
4. SEQUENCE DIAGRAMS

The Sequence Diagram models the dynamic interaction between objects over time. For this system, the most critical flow is the **Adaptive Content Request and Offline Synchronization**. Below are the various sequence diagrams:

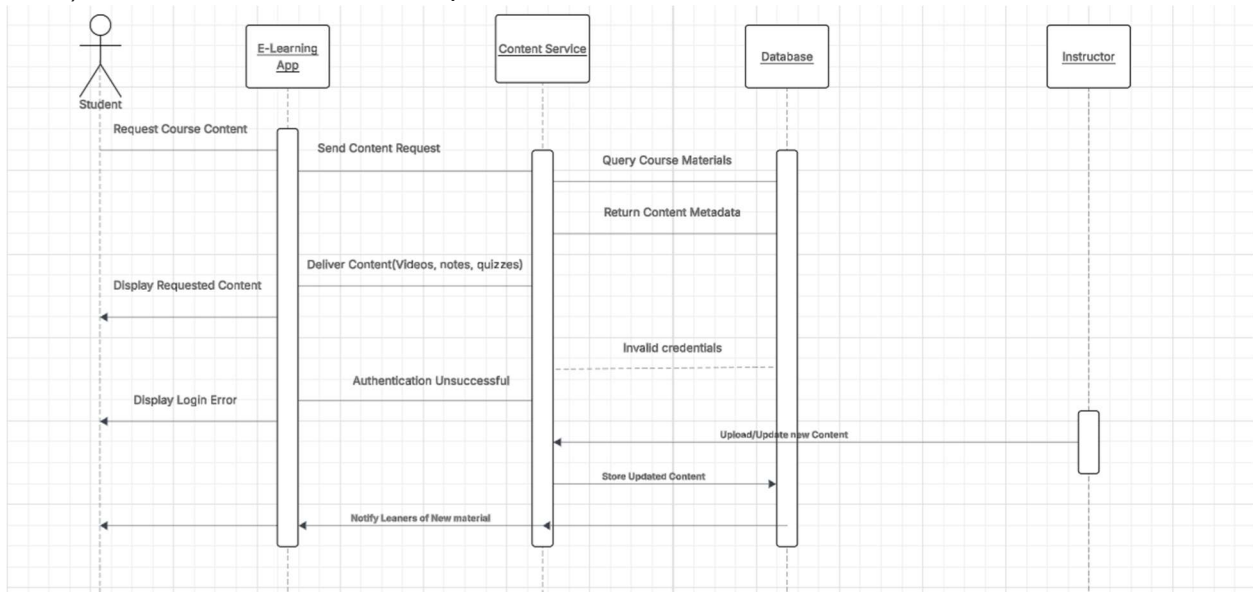
a) Attendance tracking



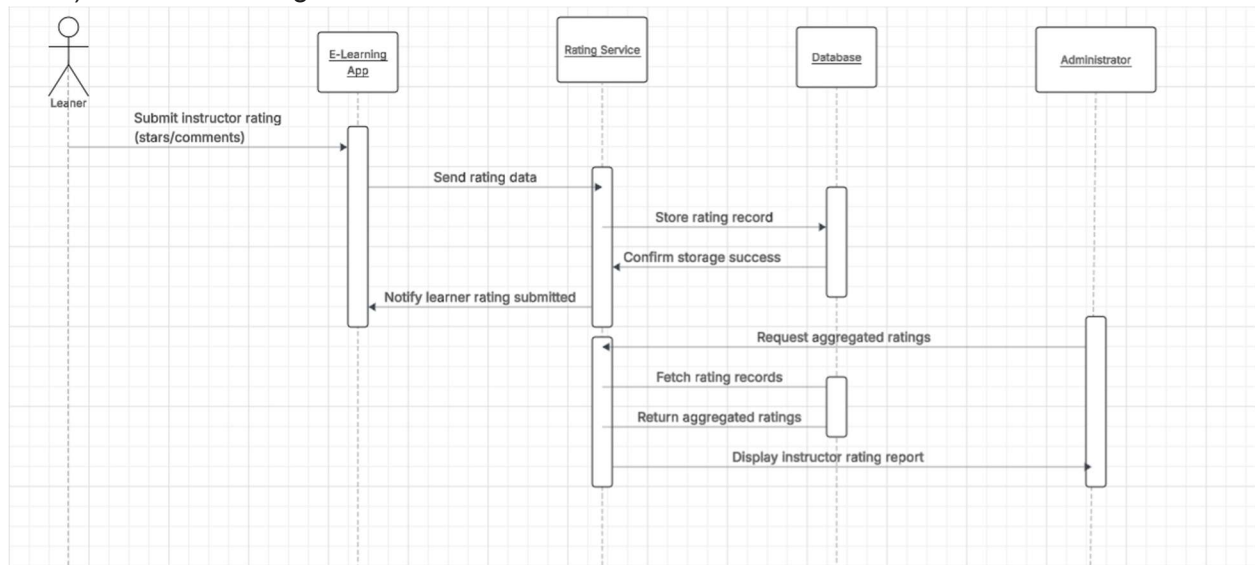
b) Authentication



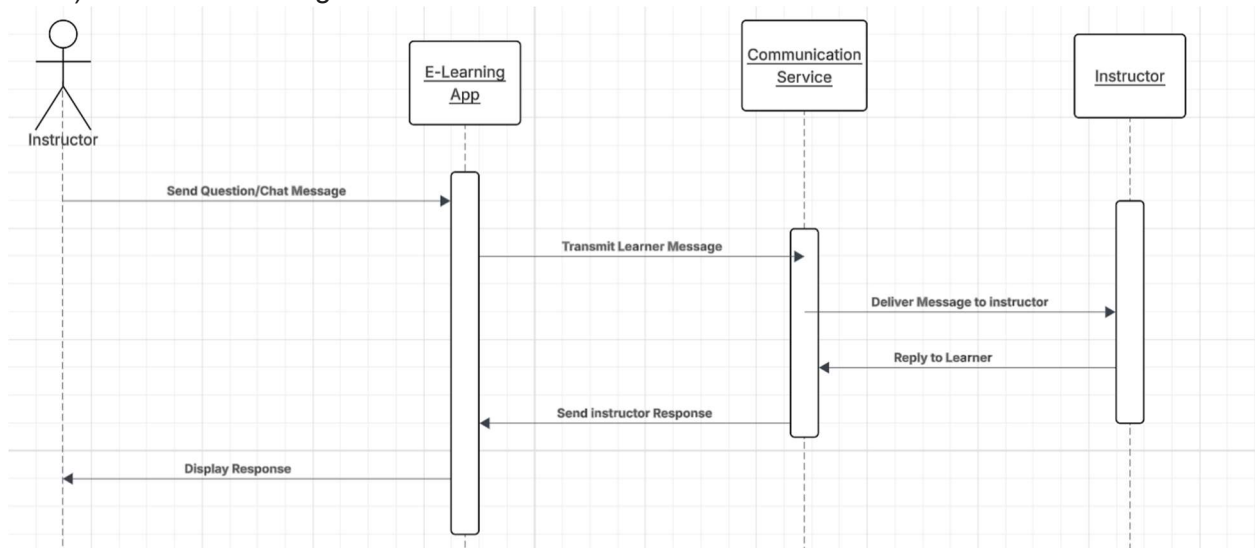
c) Course content access sequence



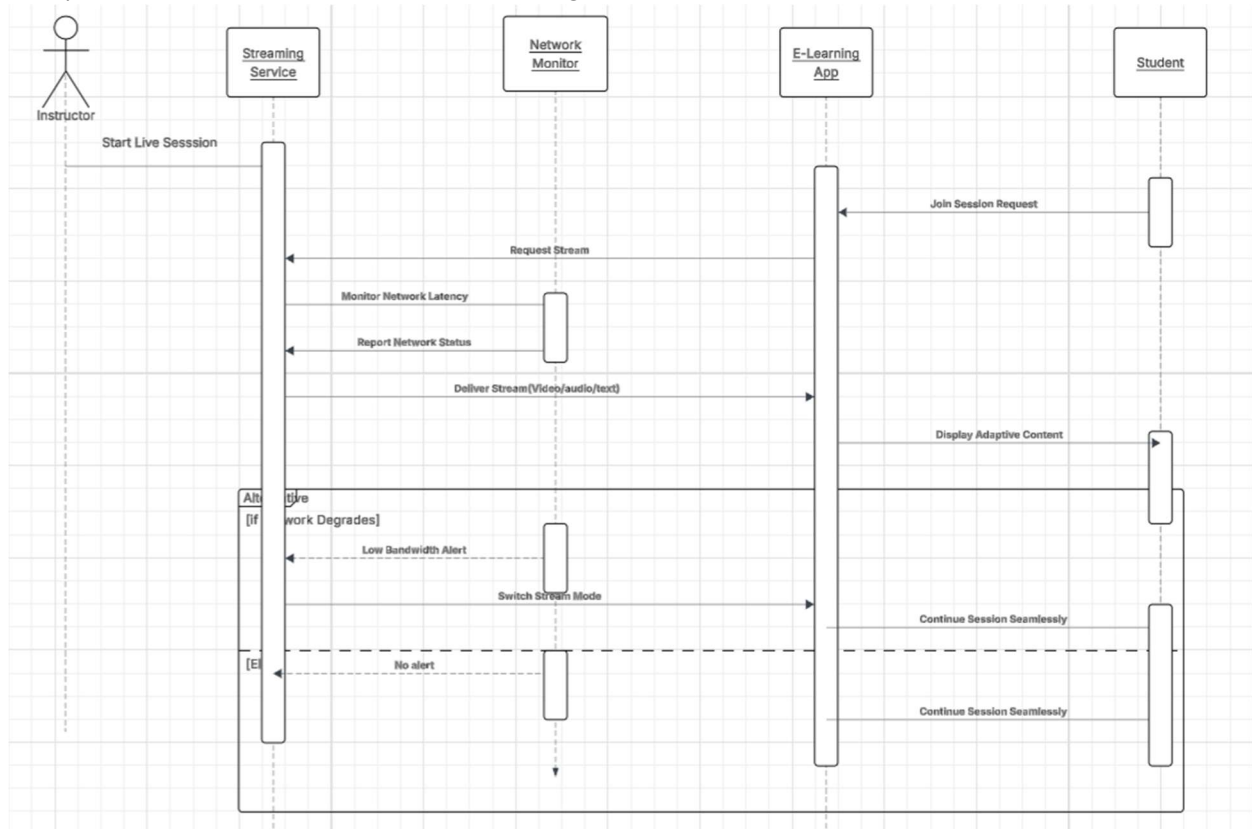
d) Instructor rating



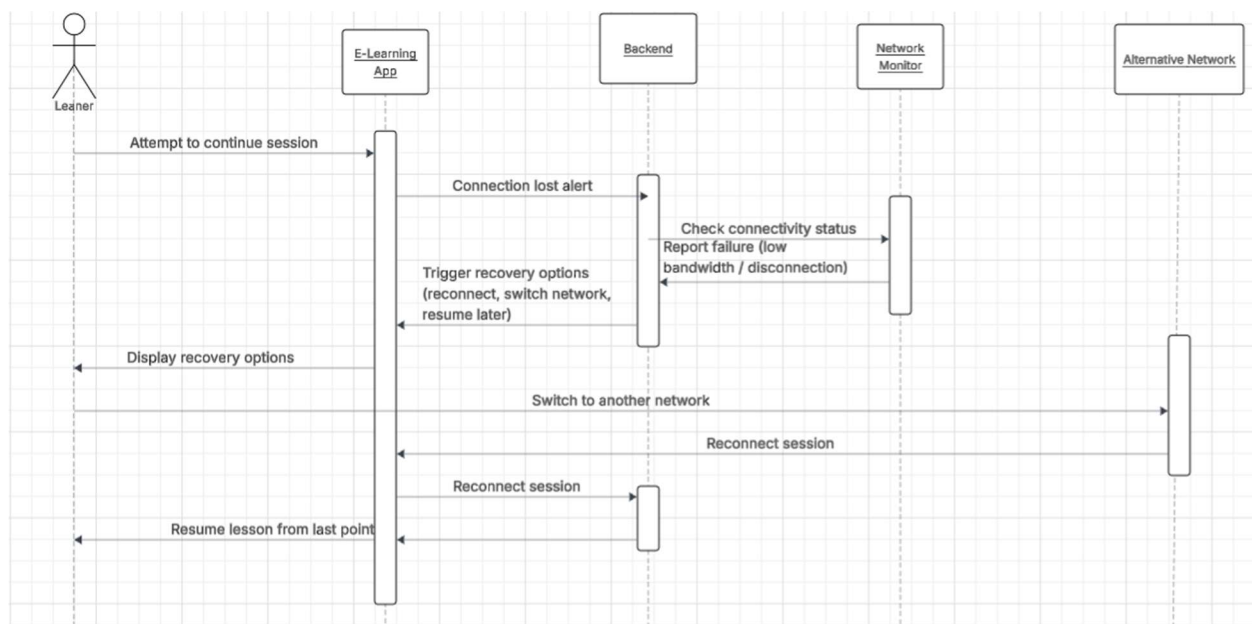
e) Interaction during lesson



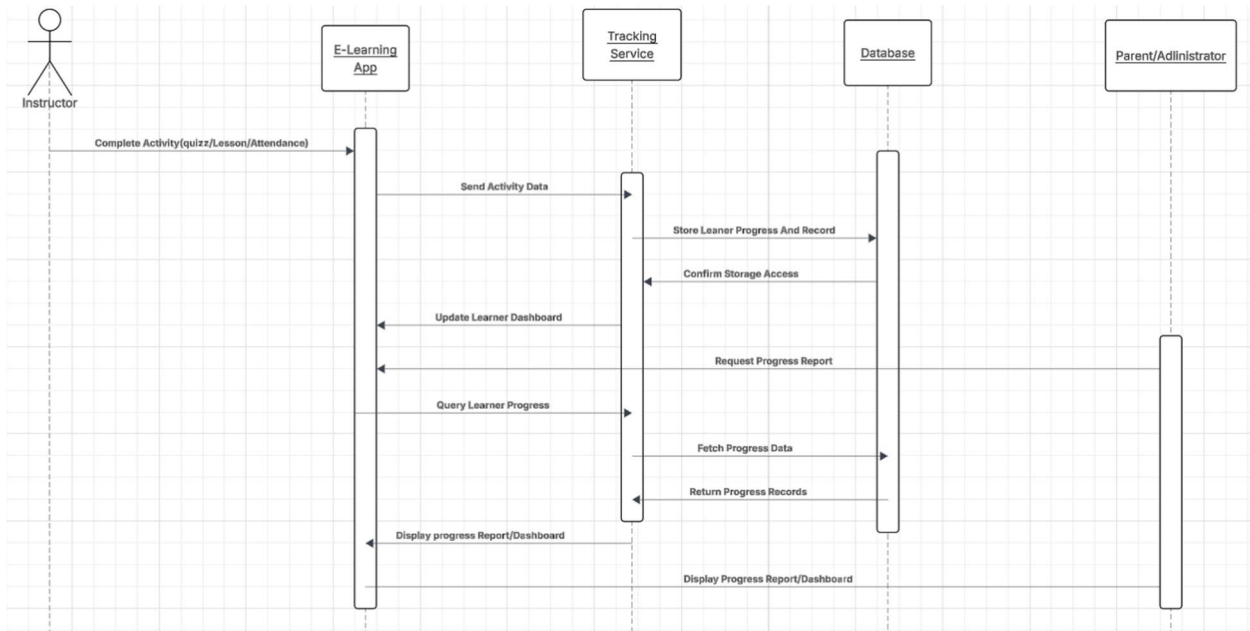
f) Live session with adaptive streaming



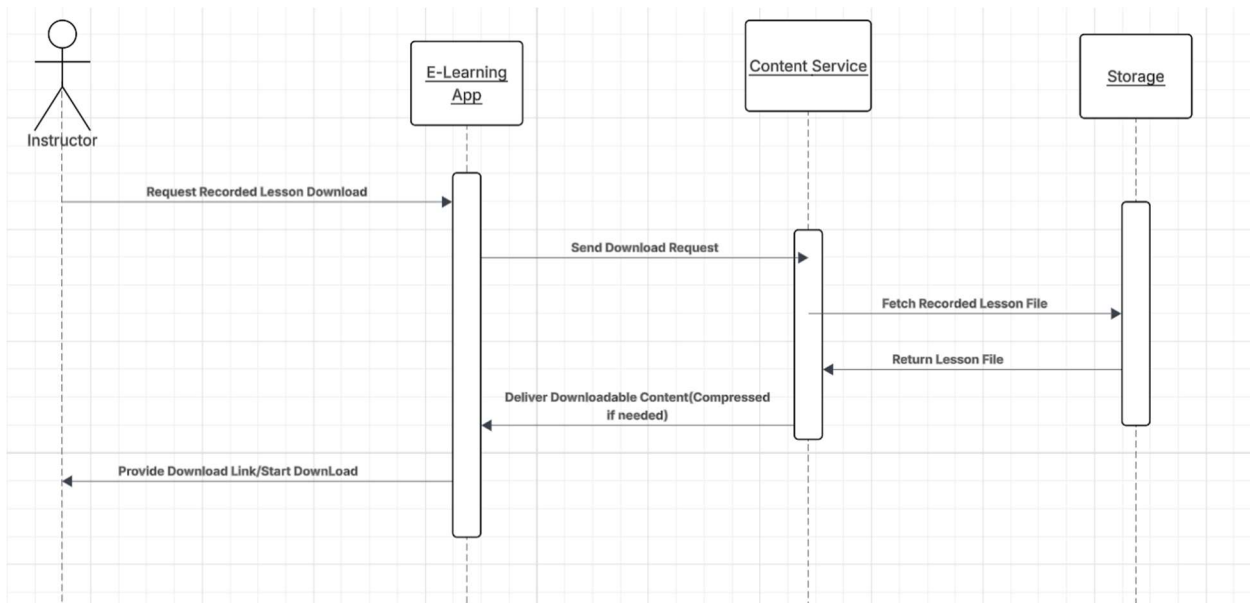
g) Network failure handling

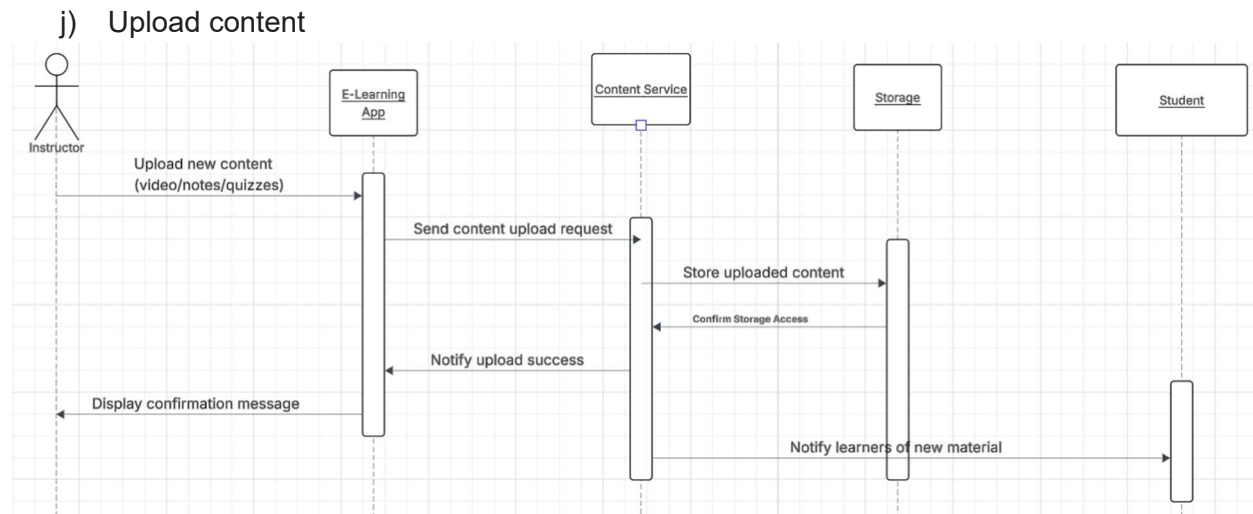


h) Progress tracking



i) Recorded lesson download





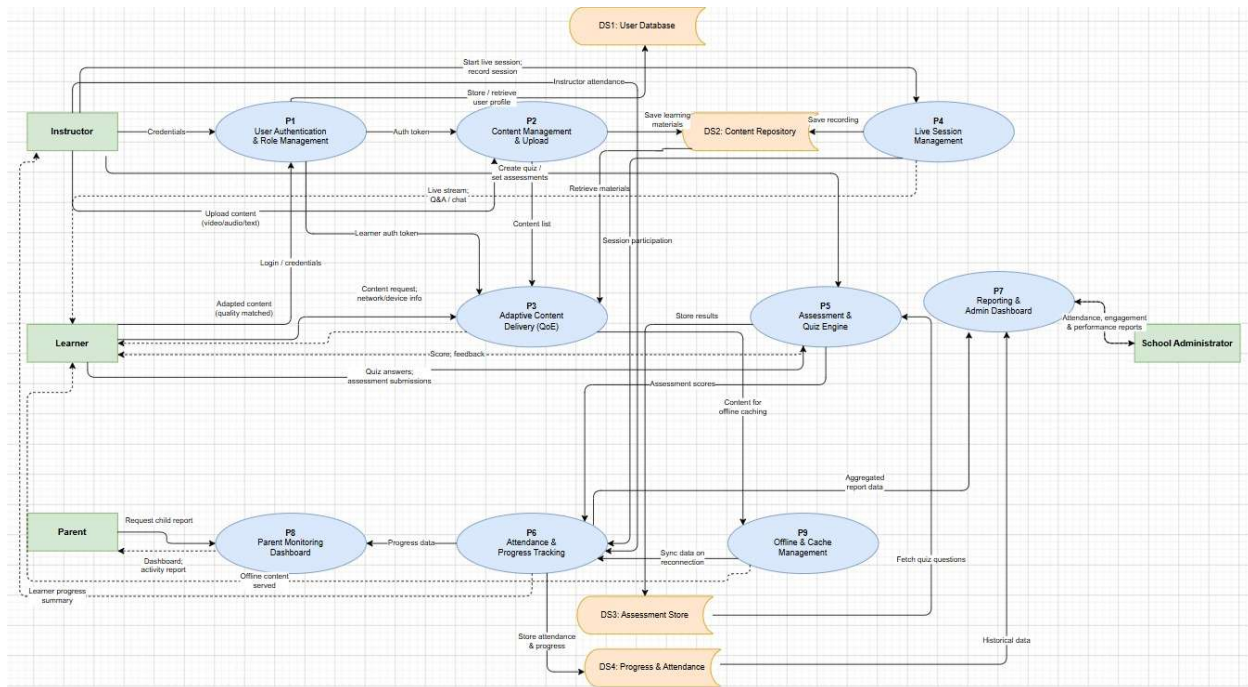
4.1 Execution Flow: Learner Accessing Content (Online vs. Offline)

1. **Initiation:** The Learner clicks to view a specific Course Module on the React PWA Frontend.
2. **Interception:** The UI sends a request that is immediately intercepted by the PWA Service Worker.
3. **Offline Scenario:** If the Service Worker detects that the device has no internet connection (e.g., due to a power outage), it queries the local Indexed DB cache. It retrieves the previously downloaded text and low-resolution materials and serves them directly back to the UI, allowing uninterrupted learning.
4. **Online/Adaptive Scenario:** If the connection is active, the Service Worker forwards the request to the Node.js/Express Backend, appending the device's current NetworkQuality status.
5. **Database Query:** The Backend queries the PostgreSQL database for the requested module.
6. **QoE Adaptation:** Before sending the response, the Backend evaluates the network status and selects the appropriate payload (e.g., stripping out heavy high-resolution videos in favor of compressed audio or text).
7. **Caching & Delivery:** The Backend returns the adapted payload to the Service Worker. The Service Worker silently updates the Indexed DB cache with this new data for future offline use, and then forwards the payload to the React UI to be displayed to the Learner.

5. DATA FLOW DIAGRAM (LEVEL 1)

5.1 Overview

The Level-1 DFD decomposes the Adaptive E-Learning Platform into nine functional processes (P1–P9) and four data stores (DS1–DS4). Each process corresponds to a cluster of related functional requirements identified in the SRS.



5.2 Data Stores

- **DS1 (User Database):** Stores user profiles, credentials, roles, and permissions. It is connected to P1 for read/write operations.
- **DS2 (Content Repository):** Stores video, audio, text, slides, and recorded sessions. It is connected to P2 (write), P3 (read), and P4 (write).
- **DS3 (Assessment Store):** Stores quiz questions, learner answers, and scores. It is connected to P5 for read/write operations.
- **DS4 (Progress & Attendance):** Stores session logs, quiz results, attendance records, and activity history. It is connected to P6 (write) and P7 (read).

5.3 Process Descriptions

- **P1 — User Authentication & Role Management:** Receives login credentials from Learners and Instructors, verifies them against DS1 (User Database), and issues authenticated tokens. It implements role-based access control (RBAC) so each user type

only accesses permitted features. Basis: SRS Functional Requirement 1 (FR1) and Non-Functional Requirement 9 (NFR9 — security).

- **P2 — Content Management & Upload:** Allows authenticated Instructors to upload learning materials in multiple formats (video, audio, text, slides). It stores all materials in DS2 (Content Repository) and makes the content catalogue available to P3 for delivery. Basis: FR2, FR3.
- **P3 — Adaptive Content Delivery (QoE):** The core differentiating process. It receives content requests from Learners together with real-time network quality signals and device capability information. It retrieves materials from DS2 and adapts the delivery quality (video resolution, compression, text-only fallback) based on bandwidth. It outputs adapted content to the Learner and also feeds P9 (offline caching) with downloadable versions. Basis: FR4, FR5, NFR3, NFR4, NFR9.
- **P4 — Live Session Management:** Manages real-time teaching sessions initiated by Instructors. It provides live video streaming and Q&A/chat to Learners, saves session recordings to DS2 for later access, and records participation data to forward to P6 (Attendance & Progress Tracking). Basis: FR9, FR10, FR11.
- **P5 — Assessment & Quiz Engine:** Allows Instructors to create quizzes stored in DS3. It serves quiz questions to Learners, receives submitted answers, scores them, returns feedback to the Learner, and forwards scores to P6 for progress tracking. Basis: FR6.
- **P6 — Attendance & Progress Tracking:** Aggregates data from three sources — session participation (P4), assessment scores (P5), and offline-sync data (P9) — and writes comprehensive progress and attendance records to DS4. It produces progress summaries for Instructors and feeds P7 (reporting) and P8 (parent dashboard). It also accepts direct attendance marks from Instructors. Basis: FR7, FR8, FR10.
- **P7 — Reporting & Admin Dashboard:** Reads historical data from DS4 and produces attendance, engagement, and performance reports for the School Administrator. It accepts report configuration commands from the Administrator. Basis: FR14.
- **P8 — Parent Monitoring Dashboard:** Receives consolidated progress data from P6 and presents it to Parents as a read-only dashboard showing attendance, quiz results, and activity summaries. Basis: FR13.
- **P9 — Offline & Cache Management:** Receives content packages from P3 and stores them in the device cache (service worker / local storage). It serves cached content to the Learner when the network is unavailable. When connectivity is restored, it synchronises learning activity data with P6 so that progress made offline is recorded. Basis: FR5, NFR5 (offline access).

5.4 Key Internal Data Flows

- **Auth token:** Travels from P1 to P2 / P3, containing verified user identity and role data.
- **Content list:** Travels from P2 to P3, containing the available materials catalogue.
- **Session participation:** Travels from P4 to P6, containing session attendance logs.
- **Assessment scores:** Travels from P5 to P6, containing quiz/test results per learner.
- **Progress data:** Travels from P6 to P7 / P8, containing aggregated progress summaries.
- **Content for offline caching:** Travels from P3 to P9, containing downloadable content

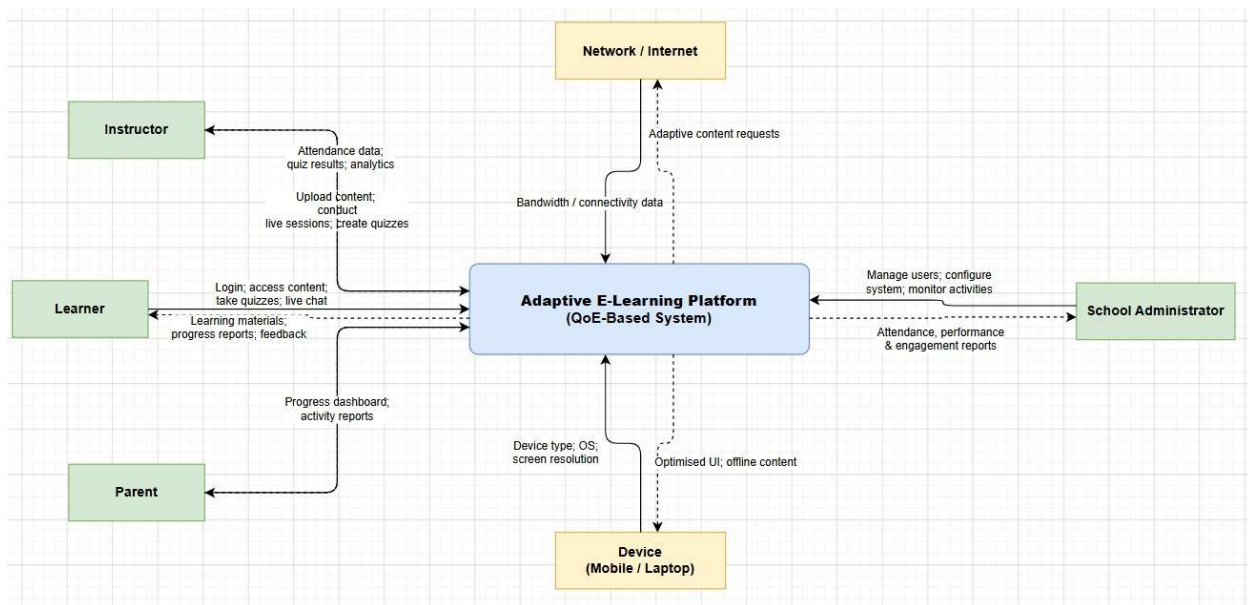
packages.

- **Sync data on reconnection:** Travels from P9 to P6, containing offline activity logs.

6. CONTEXT DIAGRAM DESCRIPTION

6.1 Overview

The context diagram for the Adaptive E-Learning Platform identifies six external entities interacting with the system. Four are stakeholders (Learner, Instructor, Parent, School Administrator) and two are infrastructure entities (Network/Internet and Device). The system occupies the center, labelled as the Adaptive E-Learning Platform.



6.2 External Entities and Their Data Flows

- **Learner:** Inputs login credentials, content requests, quiz answers, chat messages, and network/device info to the system. The system outputs adapted learning content, progress reports, quiz scores, live streams, and feedback to the Learner. This is based on FR1, FR3, FR4, FR6, and FR11.
- **Instructor:** Inputs login credentials, uploaded content (video/audio/text), live session starts commands, quiz creations, and attendance marks. The system outputs attendance data, quiz results, learner progress summaries, and analytics to the instructor. This is based on FR1, FR2, FR3, FR7, FR8, and FR12.
- **Parent:** Inputs requests for child progress reports. The system outputs a progress dashboard and activity summary to the Parent. This is based on FR13.
- **School Administrator:** Inputs user management commands and report configurations. The system outputs attendance, engagement, and performance reports to the Administrator. This is based on FR14 and FR1.
- **Network / Internet:** Inputs bandwidth and connectivity data into the system. The system outputs adaptive content requests (quality-adjusted) back to the network. This is based on

NFR3 and NFR9.

- **Device (Mobile/Laptop):** Inputs device type, OS, and screen resolution to the system. The system outputs an optimized UI and offline content to the device. This is based on NFR1, NFR2, and NFR6.

6.3 Design Decisions

Network and Device were explicitly added as external entities because the SRS identifies QoE-based adaptation as a core requirement. The system must continuously receive network quality signals and device capability data to function correctly. The Learner entity has the largest number of data flows because survey findings showed learners are the primary users across all interaction types, including content access, quizzes, live sessions, offline access, and Q&A. Separate dashed arrows are used for outputs to distinguish responses from requests, thereby improving the diagram's overall readability.

CONCLUSION

In Task 4, we successfully translated the validated functional and non-functional requirements into a comprehensive set of system models and architectural designs. By developing detailed Use Case, Class, Deployment, Sequence, Data Flow, and Context diagrams, we have established a robust structural and behavioral blueprint for the Adaptive E-learning Platform. The models strictly enforce our technological constraints. The deployment and sequence models clearly outline how the Progressive Web App (PWA) utilizes Service Workers and local database caching to guarantee access to materials during power outages. The Data Flow and Context diagrams accurately map how the backend will evaluate bandwidth and device limits to adaptively stream content. Ultimately, these diagrams provide a clear, unambiguous, and code-ready foundation for developers to transition smoothly into the physical implementation and coding phase of the project.